

1. A computer system has the main memory of size 150MB, which has been allotted to three processes  $P_1$ ,  $P_2$  and  $P_3$ . As shown in below, each process has been allocated some size of main memory, and the maximum size of main memory needed by each process is also given.

It is assumed that: (1) the main memory having been allocated to a process cannot be preempted by the other processes; (2) swapping in and swapping out of processes are not allowed.

| Process | <u>Maximum Memory Needed</u> | <u>Memory Allocated</u> |
|---------|------------------------------|-------------------------|
| $P_1$   | 70MB                         | 45MB                    |
| $P_2$   | 60MB                         | 40MB                    |
| $P_3$   | 60MB                         | 15MB                    |

When another process  $P_4$  enters into the system, it immediately requests main memory of the size 25MB, and its maximum size of main memory needed is 60MB.

Answer the following questions, by means of the banker's algorithm:

- (1) What are the contents of the matrix *Allocation*, *Max*, *Need* and the vector *Available*?
- (2) Can the  $P_4$ 's request of main memory be granted immediately? Will be the system in a safe state? and why?

Answers:

1)

| Process | <i>Max</i> | <i>Allocated</i> | <i>Need</i> | <i>Available</i> |
|---------|------------|------------------|-------------|------------------|
| $P_1$   | 70MB       | 45MB             | 25          | 25               |
| $P_2$   | 60MB       | 40MB             | 20          |                  |
| $P_3$   | 60MB       | 15MB             | 45          |                  |
| $P_4$   | 60MB       | 25MB             | 35          |                  |

Or

| Process | <i>Max</i> | <i>Allocated</i> | <i>Need</i> | <i>Available</i> |
|---------|------------|------------------|-------------|------------------|
| $P_1$   | 70MB       | 45MB             | 25          | 50               |
| $P_2$   | 60MB       | 40MB             | 20          |                  |
| $P_3$   | 60MB       | 15MB             | 45          |                  |
| $P_4$   | 60MB       | 0                | 60          |                  |

2)

It can be granted immediately.

In a safe state.

Have safe sequence  $P_1$   $P_3$   $P_4$   $P_2$

2 假设有三个美食家和一个供应者。每个美食家不停地包饺子并吃饺子，但是要包起一个韭菜鸡蛋饺子，美食家需要三种材料：韭菜、鸡蛋和饺子皮。三个美食家中，第一个拥有韭菜、第二个拥有鸡蛋、第三个拥有饺子皮。供应者无限地提供这三种材料，每次将两种不同的材料放到桌子上，拥有剩下那种材料的美食家将会包一个饺子并吃掉它，并给供应者一个信号告诉完成了，供应者就会继续放两种材料在桌上，让一位美食家包饺子并吃饺子，这种过程一直重复下去。

采用信号量，给出信号量初值，给出相关进程描述。

**Suppose there are three food enthusiasts and one supplier. Each food enthusiast keeps making and eating dumplings, but to make a chive and egg dumpling, a food enthusiast needs three ingredients: chives, eggs, and dumpling wrappers. Among the three food enthusiasts, the first has chives, the second has eggs, and the third has dumpling wrappers. The supplier provides these three ingredients infinitely, placing two different ingredients on the table each time. The food enthusiast who has the remaining ingredient will make a dumpling, eat it, and send a signal to the supplier to indicate completion. The supplier will then continue placing two ingredients on the table, allowing a food enthusiast to make and eat dumplings, and this process repeats indefinitely.**

**Using semaphores, provide the initial values of the semaphores and describe the relevant processes.**

### **Semaphore Definitions and Initial Values:**

- semaphore S = 1 // Controls mutually exclusive access to the ingredients on the table
- semaphore leek = 0 // Semaphore for available chives
- semaphore egg = 0 // Semaphore for available eggs
- semaphore wrapper = 0 // Semaphore for available dumpling wrappers
- semaphore finish = 0 // Semaphore for food enthusiast completion notification

// The semaphores representing ingredients can be replaced by pairwise combinations.

### **Process Descriptions:**

#### **Supplier Process:**

// c

while (true) {

    P(S); // Occupy the table

    // Randomly place two ingredients (one of three combinations)

    if (random number % 3 == 0) {

        V(leek); // Provide chives

```

        V(egg);    // Provide eggs
    } else if (random number % 3 == 1) {
        V(leek);    // Provide chives
        V(wrapper); // Provide dumpling wrappers
    } else {
        V(egg);    // Provide eggs
        V(wrapper); // Provide dumpling wrappers
    }

    P(finish); // Wait for food enthusiast to complete (can be
placed at the beginning of the code)

    V(S); // Release the table
}

```

**Food Enthusiast 1 (has chives) Process:**

```

// c

while (true) {
    P(egg);    // Wait for eggs
    P(wrapper); // Wait for dumpling wrappers
    P(S);      // Occupy the table (can swap order with the above)
    // Make and eat the dumpling
    V(finish); // Notify supplier of completion
}

```

```
        V(S);          // Release the table
    }
```

**Food Enthusiast 2 (has eggs) Process:**

```
// c

while (true) {

    P(leek);          // Wait for chives

    P(wrapper); // Wait for dumpling wrappers

    P(S);             // Occupy the table

    // Make and eat the dumpling

    V(finish); // Notify supplier of completion

    V(S);            // Release the table

}
```

**Food Enthusiast 3 (has dumpling wrappers) Process:**

```
// c

while (true) {

    P(leek);          // Wait for chives

    P(egg);           // Wait for eggs

    P(S);             // Occupy the table

}
```

```
// Make and eat the dumpling  
V(finish); // Notify supplier of completion  
V(S);      // Release the table  
}
```